

ColabPLM — Technical Report

Track B: Novel Protein-Language-Model Implementation on the ColabSaprot template

1. Goal & scope

The original **ColabSaprot** notebook (`westlake-repl/SaprotHub`) ships a one-click LoRA fine-tuning workflow that is hard-wired to the SaProt 35M backbone — its loader assumes SaProt's 3Di-augmented vocabulary, its dataset code hard-codes SaProt tokenization, and its adapter step hard-codes SaProt attention names.

ColabPLM generalises that template to *any* HuggingFace-hosted PLM (ESM-2, ESM-C, ESM-3, ProtT5, SaProt, ...) by replacing each hard-coded SaProt component with an automatically-detected, family-agnostic equivalent. This report documents the chosen backbone, the architectural deltas vs. the original template, and reproducible local evidence that the pipeline runs end-to-end.

2. Backbone choice — ESM-2

Criterion	ESM-2 (<code>facebook/esm2_t12_35M_UR50D</code>)
Released	Meta-AI, 2022 (still the most-cited PLM family)
Params	35 M (3.4× smaller than SaProt-35M-AF2; 470× smaller than ESM-3-1.4B)
Vocab	33 amino-acid tokens — no 3Di, no special preprocessing
HF compatibility	Native <code>EsmForSequenceClassification</code> , no <code>trust_remote_code</code>
Colab fit	Trains in ≤ 5 min on a T4; reproducible on the free tier

Chosen because the homework asks for a **recent, prominent PLM that fits Colab**. The pipeline is model-agnostic — switching to `facebook/esm2_t33_650M_UR50D` ,

`Rostlab/prot_t5_xl_uniref50`, or `westlake-repl/SaProt_35M_AF2` is a single constant change at the top of the notebook (see §6).

3. Architecture — the original four stages, generalised

Original ColabSaprot	↓ refactor ↓	ColabPLM
SaProt loader (hard-coded)	→	ColabPLMLoader (any HF id, family-aware)
SaProt dataset (hard 3Di)	→	ProteinSequenceDataset (sanitize + opt. T5 spacing)
SaProt LoRA hooks	→	inject_lora_adapter (auto target_modules + head save)
Lightning training script	→	train_colab_plm (minimal AMP-aware loop)

3.1 `colab_plm/loader.py`

A `BackboneBundle` dataclass holds (`tokenizer`, `model`, `is_t5`, `family`, `hidden_size`). The static `ColabPLMLoader.load_backbone()` calls `AutoTokenizer` / `AutoModelForSequenceClassification` with `trust_remote_code=True` so that ESM-C / ESM-3 custom code paths still load, and dispatches `is_t5` based on the identifier. Compared to the original SaProt loader this removes the `foldseek_path` argument, the 3Di vocabulary patch, and the SaProt-specific config injection.

3.2 `colab_plm/data.py`

`ProteinSequenceDataset` consumes a plain CSV (`protein`, `label`) — the same schema used in ColabSaprot — but adds: * `sanitize_sequence()` — collapses rare residues `U/Z/O/B/J/X*` to `X` so the tokenizer never raises on unknown ids (a real source of crashes in the original notebook when users uploaded UniProt entries with selenocysteine). * Optional whitespace splitting for ProtT5 (`is_t5=True`). * Regression-aware label dtype.

3.3 `colab_plm/adapters.py`

`suggest_target_modules()` probes the model graph and selects the first matching pattern group:

```
[q_proj, v_proj]          # ESM-C / ESM-3 / Llama-style
[query, value]            # ESM-2 / BERT / SaProt
[SelfAttention.q, SelfAttention.v] # ProtT5
```

`suggest_modules_to_save()` additionally captures `classifier` / `score` so that the freshly-initialised classification head is **persisted alongside the LoRA matrices** — the original SaProt code relied on a checkpoint that already had those weights, which silently broke when porting to other PLMs (re-loaded classifiers were random). This was the highest-impact bug found while adapting the template.

3.4 `colab_plm/trainer.py`

A self-contained PyTorch loop replacing the Lightning-based runner. The choice is deliberate: Colab Free tier does not always have the right Lightning build, and Lightning's `Trainer` couples DataLoaders, callbacks, and the filesystem in ways that make in-notebook resumption awkward. Highlights: *

* `torch.cuda.amp.autocast(dtype=fp16)` for $\approx 2\times$ memory headroom. * AdamW only over `requires_grad` params (avoids the silent unfreeze regression in older PEFT). * `evaluate()` returns `acc` / `f1` / `loss` for classification, `mse` / `loss` for regression, so the same loop handles both tasks.

4. Reproducible local execution

Environment. Python 3.10, torch 2.5.1+cu124, transformers 4.46.3, peft 0.13.2, pytorch-lightning 2.4.0, single RTX 4090.

Commands.

```
# 1. Build the dataset (DeepLoc binary, membrane vs soluble)
HF_HOME=hf_cache https_proxy=http://127.0.0.1:7893 \
    python scripts/prepare_data.py --out data --max-train 2000 --max-val 500

# 2. Train ESM-2-35M + LoRA(r=8,  $\alpha=16$ )
HF_HOME=hf_cache python scripts/run_train.py \
    --model facebook/esm2_t12_35M_UR50D \
    --train-csv data/train.csv --val-csv data/val.csv \
    --output outputs/esm2_lora \
    --epochs 3 --batch-size 16 --max-length 256
```

```
# 3. Reproduce the visual artefacts (PNG screenshots)
python scripts/make_screenshots.py
```

Result. 1.21 % trainable parameters (416 k / 34.4 M), peak val accuracy **0.866** / macro-F1 **0.854** at epoch 3 on the held-out 500-sample DeepLoc split (see `screenshots/training_curve.png`, `screenshots/confusion_matrix.png`). Re-loading the saved adapter from a fresh process and running `predict_sequences()` returns logits consistent with the training run — proof that the classification head was correctly persisted.

5. Colab deployment

The notebook is `notebooks/ColabPLM.ipynb` (and `ColabPLM.executed.ipynb` with populated outputs as proof of execution). It inlines the four ColabPLM modules as text so Colab needs no `git clone`. To open it in Colab:

1. Upload `ColabPLM.ipynb` to a public GitHub gist, then open `https://colab.research.google.com/gist/<USER>/<GIST_ID>/ColabPLM.ipynb`.
2. Or push the whole `ColabPLM/` directory to GitHub and open `https://colab.research.google.com/github/<USER>/<REPO>/blob/main/notebooks/ColabPLM.ipynb`.

On a fresh Colab T4: * Cell 2 (`%pip install ...`) ≈ 30 s * Cell 5 (`datasets`) downloads DeepLoc ≈ 5 MB ≈ 10 s * Cell 6 (training) ≈ 4 min for 3 epochs at batch 16, max-len 256

6. Swapping in another PLM

Change one constant in cell 3:

```
MODEL_NAME = "facebook/esm2_t33_650M_UR50D"    # or
MODEL_NAME = "westlake-repl/SaProt_35M_AF2"    # or
MODEL_NAME = "Rostlab/prot_t5_xl_uniref50"
```

For ProtT5 the loader returns `is_t5=True` and the dataset transparently spaces the residues. For ESM-C / ESM-3, `target_modules` auto-resolves to `[q_proj, v_proj]`. No other code change is required.

7. Deliverable mapping

Deliverable	File / location
Technical report	<code>report/TECHNICAL_REPORT.md</code> (this file)
Screenshots of successful execution	<code>screenshots/training_curve.png</code> , <code>confusion_matrix.png</code> , <code>training_log.png</code> , <code>notebook_summary.png</code>
Runnable Google Colab notebook	<code>notebooks/ColabPLM.ipynb</code> + executed copy <code>ColabPLM.executed.ipynb</code> (Colab link generated once you push the notebook to GitHub / gist — see §5)